

Lab 4 – Clock Synchronization

Ex 1: Network Time Synchronization Using NTP in Java

Objective

In this lab, students will implement an NTP client in Java to retrieve time from an NTP (Network Time Protocol) server. They will analyze how UDP communication is used to access an NTP time server, extract timestamps, and calculate round-trip delay and clock offset to adjust the local system time.

Background

- NTP (Network Time Protocol) is used to synchronize clocks across computer networks.
- NTP operates over UDP (User Datagram Protocol), which is a connectionless protocol that allows fast communication but does not guarantee reliability.
- The standard NTP server port is 123.
- Time is returned in NTP timestamp format, where time is measured in seconds since January 1, 1900.
- The client must convert NTP time to Unix time format (seconds since January 1, 1970).
- Java's DatagramSocket and DatagramPacket classes are used to send and receive UDP packets.

Understanding the Time Values (T1, T2, T3, T4)

The NTP client uses four key timestamps to calculate synchronization parameters:

- T1 (Client Send Time): The time when the client sends the request to the NTP server.
- T2 (Server Receive Time): The time when the NTP server receives the request.
- T3 (Server Transmit Time): The time when the NTP server sends the response.
- T4 (Client Receive Time): The time when the client receives the response back from the NTP server.

Round-Trip Delay Calculation

The round-trip delay (RTT) measures the total time it takes for a request to travel to the NTP server and back. It is calculated using the formula:

$$\text{Round-Trip Delay} = (T4 - T1) - (T3 - T2)$$

Clock Offset Calculation

The clock offset determines how much the local system clock differs from the NTP server's time. A negative offset means the client clock is ahead, while a positive offset means it is behind.

$$\text{Clock Offset} = ((T2 - T1) + (T3 - T4) - \text{Round-Trip Delay}) / 2$$

Provided Code

Import the Eclipse project (**ntysync**) with the Java program, **NTPClient.java** that has been provided that:

1. Creates a UDP socket to communicate with an NTP server.
2. Sends an NTP request to retrieve the current time.
3. Receives the response and extracts the NTP timestamps.
4. Computes round-trip delay and clock offset for accurate synchronization.

However, the code is incomplete. Some parts are marked with TODO comments, which students need to implement.

Tasks

1. Analyze the Code:

- Understand how DatagramSocket and DatagramPacket are used for UDP communication.
- Review how timestamps are extracted from the NTP response.

2. Complete the Missing Code:

- Capture local timestamps (T1, T4) before and after sending the request.
- Extract timestamps (T2, T3) from the NTP response.
- Compute round-trip delay using the formula:
$$\text{Delay} = (T4 - T1) - (T3 - T2)$$
- Compute clock offset using the formula:
$$\text{Offset} = ((T2 - T1) + (T3 - T4) - \text{Delay}) / 2$$

["time.google.com"](https://time.google.com)
- Print the system time, NTP time, round-trip delay, and clock offset.

3. Test the Implementation:

- Run the program multiple times to observe variations in time and offset.
- Try querying different NTP servers (e.g., pool.ntp.org, time.apple.com).

Discussion Questions:

1. Why does NTP use UDP instead of TCP?
2. What happens if network delay is high? How does that affect synchronization?

Note: Write the answers to the questions in a text document.

Expected Output (Example)

T1 (Client Send Time): Sun Mar 10 10:30:45 UTC 2025 | Local time when request was sent.

T2 (Server Receive Time): Sun Mar 10 10:30:45 UTC 2025 | Time when server received the request.

T3 (Server Transmit Time): Sun Mar 10 10:30:45 UTC 2025 | Time when server sent the response.

T4 (Client Receive Time): Sun Mar 10 10:30:46 UTC 2025 | Local time when response was received.

NTP Time from Server: Sun Mar 10 10:30:45 UTC 2025

Round-Trip Delay: 50 ms

Clock Offset: -20 ms

Client clock is AHEAD of the NTP server by 20 ms.

Ex 2: Understanding and Implementing Vector Clocks

Objective

This exercise extends from **Lab 3** and will introduce you to vector clocks and their role in tracking causal relationships in a distributed payment system using Apache ZooKeeper. By the end of this lab, you will:

- Understand how vector clocks track event ordering in a distributed system.
- Learn how the Payment Client and Server use vector clocks to maintain a causally consistent state.
- Implement a missing functionality in the Payment Server to track how many payment requests each server has processed.
- Answer conceptual questions about vector clocks, causal order violations, and distributed consistency.

Project Setup: Open the `pmvectorclk` Project in Eclipse

This lab is based on the `pmvectorclk` Eclipse project, which contains:

- `PaymentClient.java` – Sends payment requests to servers.
- `PaymentServer.java` – Processes payment requests and acknowledges them.
- `VectorClock.java` – Implements the vector clock data structure.
- Apache ZooKeeper is used as the coordination service.

Steps to Open the Project in Eclipse

1. Open Eclipse.
2. Import the `pmvectorclk` Project:
 - Go to File → Import.
 - Select Existing Projects into Workspace.
 - Choose the `pmvectorclk` directory.
 - Click Finish.
3. Ensure ZooKeeper is running before running the Payment Server and Client.
4. You can run multiple Payment Servers and then run the Client.

How Vector Clocks Are Used in the Payment System

This payment system consists of a client and multiple servers, communicating via ZooKeeper.

Payment Client (Sends Payment Requests)

1. The client maintains a vector clock.
2. Before sending a payment request, the client increments its own counter in the vector clock.
3. The client waits for an acknowledgment (ACK) from the server.
4. When the client receives an ACK, it merges the server's vector clock into its own.

Payment Server (Processes Requests)

1. The server reads a payment request from ZooKeeper.
2. It retrieves the vector clock from the request.
3. It merges the client's vector clock into its own.
4. It increments its own counter in the vector clock.
5. The original request is deleted after processing.

Hands-on Exercise: Track Server Processing Counts Using Vector Clocks

 Your Task:

Currently, the Payment Server receives vector clocks from the client but does not print how many requests each server has processed.

 Modify the Payment Server code to print the number of payment requests processed by

each server.

TODO in `PaymentServer.java`

Find the following TODO comment in `PaymentServer.java` and implement the missing functionality:

```
```java
// ===== TODO EXERCISE FOR STUDENTS =====
// Task: Print how many payment requests each server has processed.
// 1. Iterate over the vector clock entries.
// 2. Print each server's name and the number of requests it has processed.
// 3. Use the format: " Server_101 has processed X requests."
// =====
```
```

✓ Expected Output

After implementing the TODO in `PaymentServer.java`, the expected output should be:

...



Processing payment for request: request_XYZ123



Updated vector clock: {client=4, server_101=5, server_102=3, server_103=6}



Payment requests processed by each server:



Server_101 has processed 5 requests.



Server_102 has processed 3 requests.



Server_103 has processed 6 requests.

...

Discussion Questions:

1. How does the Vector clocks help each process to track the state of each other process? Explain using the given example code.
2. How do vector clocks help prevent causal ordering violations in distributed systems? Can you explain using the given example code.

Note: Write the answers to the questions in a text document.

Submission :

Upload the updated projects with the text document with the answers in a single zip file.
The Zip file name should be your registration number.